

TagManager (Shashank):

Description:

The goal of TagManager class is to let AI agents efficiently track and query information about objects in the world using tags that are string based.

Tag Manager works entirely with object IDs instead of agents storing raw pointers or references to avoid lifetime and ownership issues when objects are created or destroyed.

TagManager is designed to integrate with AnnotationSet, which stores the tags for individual objects, while TagManager provides fast queries across all objects

Design:

Tag Manager maintains two synchronized data structures:

1. A map from object IDs to their set of tags
2. A reverse map from tags to the set of object IDs that have that tag.

This two-way structure is used to answer questions like:

1. Which tags does this object have?
2. Which objects have tag A?
3. Which objects have tag A and tag B but not tag C?

I also chose tag normalization in the design which means trimming tags of whitespace so that tags with same names but differences in whitespace are treated as the same tag. Furthermore, Whitespace-only tags are rejected as invalid.

Core API functions:

The main public functions include:

1. Register object and Unregistered object to manage object lifetimes
2. AddTag, RemoveTag, and HasTag to manipulate tags
3. Count to see how many objects have a given tag
4. FindAny, returns objects that have any of the set of tags
5. FindAll, returns objects that have all the required tags
6. FindAllExcept, supports both required and forbidden tags

-> For Performance, FindAllExcept starts from the smallest tag bucket when doing AND-style queries to reduce unnecessary checks and make the system scale better as the objects grow

AnnotationSet(Sachin)

Description

The goal of AnnotationSet is to hold all the tags that pertain to a certain object. It allows for other classes to figure out what tags are connected to which objects and to attach specific tags to objects in the world.

AnnotationSet uses object Ids for the same reason as tagmanager.

Design:

Annotation Set holds both an integer representing the Object Id and a set that contains all tags pertaining to that object.

The set allows for no duplicate tags that are connected to the same object while allowing for tag checking.

Core Api Functions:

Annotation Set's core functions are adding and deleting tags from an object and checking tag membership on that object.

RobinHoodMap (Shamar):

Description:

The goal of RobinHoodMap is to provide a fast hash map for the project, as an alternative to `std::unordered_map`.

It uses open addressing and Robin Hood hashing to keep lookup performance consistent, even as the table fills.

Design:

RobinHoodMap stores entries directly in a bucket array (open addressing).

Each bucket stores:

- the entry (key/value)
- the entry's hash
- the entry's probe distance (how far it is from its ideal bucket)

Robin Hood insertion rule:

- When inserting, if the incoming element has probed farther than the element currently in the bucket, it swaps with it.
This helps reduce long probe chains and makes lookup time more stable.

Additional design choices:

- Bucket count is always a power of two, so indexing can use `hash & mask` for speed.
- Find uses a Robin Hood early-exit rule: if a bucket's stored probe distance is smaller than our current probe distance, the key cannot appear later in the probe chain.

Core API functions:

Main public functions include:

- `Insert(key, value)` → returns `{pointer_to_value, inserted}`, where `inserted` is false if the key already existed.
- `Find(key)` → returns pointer to value or `nullptr`.
- `Contains(key)` → boolean check using `Find`.
- `At(key)` → returns reference or throws `std::out_of_range` if missing.
- `Reserve(n)` → ensures capacity for `~n` elements without exceeding max load factor.
- `Rehash(bucket_count)` → rebuilds the table at a new bucket count.
- `MaxLoadFactor(f) / MaxLoadFactor()` → controls the growth threshold.

Important usage note:

Because this is open-addressed and may rehash on insert, insertions can invalidate pointers/references/iterators.

How it might be used:

RobinHoodMap can be used for fast associative storage in the agent system, such as:

- object ID → feature data
- cached decision results
- agent memory tables
- internal indexing structures used by tag or annotation systems

FeatureVector (David)

I created the `FeatureSchema` and `FeatureVector` class.

`FeatureSchema` is like the contract. You construct it with a list of unique, non-empty feature names. For example, in our society simulation, "nearby allies", "hunger", "happiness", "trade opportunity" would be feature names. Internally it builds a name to

index map, so you can call `Name()` with an index as a parameter to get the feature name at an index, and `IndexOf("hunger")` to find where that feature is. Two schemas are equal if their name lists match exactly.

`FeatureVector` is the data. It owns a dense `values_` array sized to the schema, initialized to zeros. The api allows for a few different operations. `Clear()` zeroes everything out. You can call `Get(i) / Set(i, v)` for index-based access. You can also call `GetByName(name)` to return the value stored at that feature name. `TrySet(name, v)` sets by name and returns success/failure. And as far as numerical operations, I added a `Dot()` function that computes an inner product. I wanted to add to just keep in mind; I may be changing the naming specifics in the future.

A little bit about use cases. Agents need a clean bridge from local observations to decisions. A villager can build a feature vector from what it sees in range X, things like nearby agents, resource scarcity, recent trades, hostility signals, whatever it ends up being. Then it can score candidate actions by dotting that vector with a weight vector. In an interaction-heavy world, those social features like trust, reputation, threat, etc. become more valuable signals that could drive emergent behavior.

Team wise, ideally the world, agent, and analytics can all share one schema so everyone's

"feature number x" stays the same thing, everywhere.

Thank you

Function Set

Description

I created the Function Set to handle events. `FunctionSet` is a utility function that lets us store multiple callbacks that all share the same function signature, and then invoke them all in one call.

So instead of having one function pointer, we have a list of functions, and we can treat it like an event signal.

For example, in our simulation, when an AI agent enters a new location, several different systems might need to react:

- The memory system might record the location.
- The `TagManager` might update nearby tag-based queries.

- The UI system might display the agent's new position.
- The analytics system might log movement statistics.

Instead of hard-coding all of these reactions inside the agent class, we can expose an event using FunctionSet. I made FunctionSet templated on a function signature because the whole point of the class is: “store many callbacks that all have the same signature.” But in our project, the signature changes depending on the event.

So instead of writing separate classes like:

- FunctionSetVoid
- FunctionSetInt
- FunctionSetString
- FunctionSetAgentEven

I wrote one generic container and let the user decide the function signature through the template parameter.

You can think of FunctionSet like a small event system. Other parts of the program “sign up” by adding a function, and when something happens, we call all of them together. I kept the implementation simple and lightweight for now so it is easy to use and integrate, while still leaving room for future improvements or extensions if the project grows in complexity.

In the actual implementation of the Function Set class, we have the public functions which are:

Public Functions

- FunctionSet() — This is the constructor, Creates an empty FunctionSet with no functions inside.
- Add(FunctionType fn) — Adds a function to the list so it will be called later.
- Clear() — Removes all functions from the list.
- CallAll(Args... args) — Calls every stored function in the order they were added using the given arguments.
- Empty() — Checks whether there are currently no functions stored

Private Member

functions_(a vector) — A list that stores all the added functions so they can be called later in order.

Use Cases

FunctionSet can be useful for both the AI Agent group and the other groups in Company C. In the AI Agent group, it can be used when an agent changes state, such as moving to a new location, gaining a tag, or updating a feature, allowing multiple systems like memory, learning, UI, or analytics to react without tightly coupling them to the agent. For the World and GUI groups, it can be used to handle events like object creation, object removal, or button clicks, where several systems may need to respond at the same time. Overall, FunctionSet helps different parts of the program react to shared events in a clean and modular way. It is basically an event management system.

Thank you!